

# JDBC — Java Database Connectivity

## Software Developer Interview Questions & Answers

Comprehensive Guide | Core to Advanced

### Section 1: Core Java & JDBC Basics

#### Q1. What is JDBC? Why do we need it?

**Answer:**

JDBC stands for Java Database Connectivity. It is an API (Application Programming Interface) that allows Java programs to connect to relational databases, run SQL queries, and retrieve or manipulate data.

- Java understands Java commands; databases understand SQL — they are incompatible.
- JDBC acts as a bridge to overcome this incompatibility.
- It allows interaction with databases like MySQL, Oracle, and PostgreSQL from Java.
- It enables performing CRUD (Create, Read, Update, Delete) operations from Java code.

#### Q2. What is a JAR file? What is its purpose?

**Answer:**

JAR stands for Java Archive. It is a compressed/packaged version of Java programs.

- Contains .java files (source) and .class files (compiled/executable).
- Main purpose: sharing Java programs from one developer to another.
- Examples: String class, Object class, Collection Framework — all shared via JAR files in JRE.
- JAR files are collections of packages, also called internal libraries.

#### Q3. What is a Maven Project? Why is it used?

**Answer:**

Maven is a powerful build automation and project management tool for Java projects.

- Simplifies project setup, dependency management, building, testing, and deployment.
- Uses a file called pom.xml (Project Object Model) to store project details.
- Dependencies are external JAR files (e.g., MySQL, Hibernate, Servlet API).
- Maven automatically downloads required JAR files from the central repository.
- Downloaded files are stored in the local .m2 folder.

#### Q4. What is an API? Give examples related to Java.

**Answer:**

API stands for Application Programming Interface. It is a software containing rules and protocols that helps two incompatible software systems communicate with each other.

- It is a collection of classes and interfaces.
- Used for inter-application communication (sharing data between applications).

Examples:

- Java API → java.lang package
- Utility API → java.util package
- JDBC API → java.sql package
- Servlet API → javax.servlet / jakarta.servlet package

#### Q5. What is the difference between File System and Database?

**Answer:**

**File System:**

- Stores data in plain files (.txt, .csv, etc.).
- No built-in query processing — manual access.
- Less security, less consistency.
- Manual backup and recovery.
- Examples: Windows File Explorer, Linux File System.

**Database:**

- Stores data in structured tables (rows and columns).
- Efficient query processing using SQL.
- More security (roles, authentication, encryption).
- Automated backup, rollback, and recovery.
- Examples: MySQL, Oracle, SQL Server, PostgreSQL.

## Section 2: JDBC Architecture & Drivers

### Q6. What are the 4 components of JDBC architecture?

#### Answer:

The JDBC architecture consists of 4 components:

- Java Application — Writes source code and accepts input from user (via Scanner or HTML).
- JDBC API — Connects and executes queries with the database (java.sql package).
- JDBC Driver — convert java queries into sql queries.
- Database — Relational database where data is stored permanently (MySQL, Oracle, etc.).

### Q7. What are the 4 types of JDBC Drivers?

#### Answer:

##### Type 1 - JDBC-ODBC Bridge Driver:

- Converts JDBC calls into ODBC calls. Needs ODBC installed. Slow and outdated.

##### Type 2 - Native-API Driver (Partially Java):

- Converts JDBC calls into native DB API calls. Needs database software on client. Not portable.

##### Type 3 - Network Protocol Driver (Fully Java):

- Sends JDBC calls to a middleware server. Platform-independent. Fully written in Java.

##### Type 4 - Thin Driver (Fully Java):

- Directly connects Java to the database without middleware. Fastest and most commonly used.

### Q8. What is the most commonly used JDBC driver and why?

#### Answer:

Type 4 — Thin Driver is the most commonly used JDBC driver.

- It directly converts JDBC calls into the specific database protocol.
- Fully written in Java — no need for middleware.
- Platform-independent and fast.
- Does not require any additional client-side software.
- That is why it is known as the 'thin' driver.

### Q9. What is DriverManager? What is its role?

#### Answer:

DriverManager is a helper class present inside the java.sql package.

- Manages database drivers and establishes connections between Java and database.
- Acts as a bridge between the Java program and the database.

- Selects the correct driver based on the database URL.
- Key method: `DriverManager.getConnection(url, user, password)`

## Section 3: JDBC Steps & Connection

### Q10. What are the steps to connect Java application to a database using JDBC?

**Answer:**

Step 1: Load and Register the Driver (not required from JDBC 4.0+ as it auto-loads).

Step 2: Establish the Connection between Java and Database using `DriverManager.getConnection()`.

Step 3: Create a Statement / PreparedStatement / CallableStatement.

Step 4: Execute the Query using `execute()`, `executeQuery()`, or `executeUpdate()`.

Step 5: Close the Connection — mandatory to free resources.

### Q11. How to load and register a JDBC driver?

**Answer:**

- Use `Class.forName("com.mysql.cj.jdbc.Driver");`
- It is a static method in the Class class that loads the driver by its fully qualified name.

Note: From JDBC 4.0 / Java 6+, driver registration is automatic when the JAR is added.

### Q12. How to establish a database connection using `getConnection()`?

**Answer:**

**Method 1 — `getConnection(url, username, password)`:**

- Accepts three String arguments: URL, username, password.
- Most commonly used approach.
- Example: `"jdbc:mysql://localhost:3306/dbName","root","sql123"`.

### Q13. What is a database URL in JDBC? What does it contain?

**Answer:**

URL stands for Uniform Resource Locator. It identifies and accesses the database from Java.

Syntax: `protocol//hostinfo:portnumber/dbname?userinfo`

- Protocol: `jdbc:mysql` (for MySQL)
- Host: `localhost`

- Port Number: 3306 (MySQL default)
- Database Name: your\_database\_name

Example: jdbc:mysql://localhost:3306/myDatabase

## Section 4: Statement, PreparedStatement & CallableStatement

### Q14. What is a Statement in JDBC? When should you use it?

#### Answer:

Statement is an interface in the java.sql package used to execute SQL queries.

- Obtained via: `con.createStatement()`
- Used to execute static queries — queries that do not change at runtime.
- Suitable for DDL commands: CREATE, ALTER, DROP, TRUNCATE.
- Not recommended for INSERT, UPDATE, DELETE due to security risks.

### Q15. What are the drawbacks of using Statement?

#### Answer:

- Not Secure — Vulnerable to SQL Injection attacks.
- No Pre-Compilation — Query is compiled every time it runs (slower).
- Slow for Repeated Queries — Same query running multiple times is inefficient.
- No Type Safety — Cannot set specific data types (int, String, Date) directly.

### Q16. What is PreparedStatement? How is it different from Statement?

#### Answer:

PreparedStatement is a sub-interface of Statement in java.sql. It is used to execute dynamic queries — queries that include user input at runtime.

- Uses placeholders (?) for parameters: `INSERT INTO products VALUES (?, ?, ?, ?)`
- Created via: `con.prepareStatement(String query)`
- Uses setter methods: `setInt()`, `setString()`, etc.

#### Key Differences:

- PreparedStatement is precompiled — faster for repeated queries.
- More secure — prevents SQL injection.
- Suitable for DML: INSERT, UPDATE, DELETE.
- Can store/retrieve images and files (CLOB/BLOB).

### Q17. What is CallableStatement? When is it used?

#### Answer:

CallableStatement is an interface used to call and execute Stored Procedures and Functions in a database.

- It is a child interface of both Statement and PreparedStatement.
- Created via: `con.prepareCall(String sql)`
- Used when complex business logic is stored in the database as procedures.
- Supports IN, OUT, and INOUT parameters.
- For OUT parameters: `registerOutParameter()` must be called before execution.

### Q18. What are the 3 methods used to execute queries in JDBC?

#### Answer:

#### 1. `boolean execute(String sql):`

- Executes any type of SQL (DDL, DML, DQL).
- Returns true if result is a ResultSet (DQL), false for update count.
- Use `getResultSet()` to retrieve the ResultSet.

#### 2. `ResultSet executeQuery(String sql):`

- Executes only SELECT (DQL) queries.
- Returns a ResultSet object containing the query results.

#### 3. `int executeUpdate(String sql):`

- Executes only DML queries: INSERT, UPDATE, DELETE.
- Returns the number of rows affected.

## Section 5: ResultSet, Metadata & Batch Processing

### Q19. What is a ResultSet in JDBC?

#### Answer:

ResultSet is the output of a SELECT query. When a database executes a SELECT query, it stores a copy of the matched records in buffer memory — this is the ResultSet.

Key methods:

- `next()` — Shifts cursor to next row; returns true if record exists, false if not.
- `getters()` — Methods like `getInt()`, `getString()` to fetch column values.
- `isBeforeFirst()` — Checks whether the first record is present in ResultSet.

ResultSet object can be obtained via:

- `statement.getResultSet()`

- `statement.executeQuery()` — directly returns `ResultSet`.

## Q20. What is Batch Processing in JDBC? When should it be used?

### Answer:

Batch Processing is a mechanism that groups multiple SQL statements and executes them all at once instead of one-by-one.

- Uses `executeBatch()` which returns `int[]` — each index has the affected row count.
- Significantly improves performance for bulk data operations.
- Ideal when inserting, updating, or deleting large amounts of data.

Allowed operations in batch: INSERT, UPDATE, DELETE (DML).

Not allowed: DDL (not recommended), DQL (SELECT), TCL, DCL.

## Section 6: Stored Procedures, Functions & Connection Pooling

## Q21. What are Stored Procedures? Why are they used?

### Answer:

Stored Procedures are groups of SQL statements saved permanently in the database to perform specific tasks.

- Called using CALL statement.
- Support IN, OUT, and INOUT parameters.
- Can perform transaction control (COMMIT, ROLLBACK).

### Why use Stored Procedures?

- Performance — Precompiled, execute faster.
- Reusability — Write once, call from Java, .NET, Python, etc.
- Security — Users access procedure without seeing tables directly.
- Reduce Network Traffic — One call instead of many SQL statements.
- Maintainability — Change procedure without updating application code.

## Q22. What is Connection Pooling in JDBC? Why is it important?

### Answer:

Connection Pooling is a collection of pre-created, reusable database connection objects. Instead of creating a new connection for every operation, connections are taken from the pool and returned after use.

### Why is it important?

- Improves Performance — Reuses existing connections; no overhead of creating new ones.
- Reduces Creation Overhead — Creates connections once and reuses them.
- Better Resource Management — Limits total connections to the database.
- Faster Response Time — Connections are immediately available.
- Improves Scalability — Multiple users can be served simultaneously.
- Better Stability — Prevents database crashes from too many connections.

### Q23. What is Transaction Management in JDBC?

#### Answer:

A transaction is a group of multiple related operations treated as a single unit.

- If ALL operations succeed → data is committed permanently (COMMIT).
- If ANY operation fails → all changes are reverted (ROLLBACK).

Example — Flight Booking:

- Select route → Search flights → Select flight → Enter details → Payment
- If payment fails, all previous steps must be rolled back.

Purpose: Prevents junk/incomplete data from being stored in the database.

By default, JDBC auto-commits each statement. Use `con.setAutoCommit(false)` to manage transactions manually.

## Section 7: Scenario-Based & Advanced Questions

### Q24. What is SQL Injection? How does PreparedStatement prevent it?

#### Answer:

SQL Injection is a security attack where malicious SQL code is inserted into user input fields to manipulate queries.

Example Attack (using Statement):

- Input: ' OR '1'='1 → bypasses login validation entirely.

How PreparedStatement prevents it:

- Uses placeholders (?) instead of directly embedding user input.
- User input is treated as a literal value, not executable SQL code.
- Parameterized queries are precompiled — structure cannot be changed by input.



### Q25. What happens if you forget to close the database connection?

#### Answer:

Forgetting to close a database connection can cause serious problems:

- Connection Leak — Open connections accumulate and consume memory.
- Database Overload — Too many open connections crash or slow the database.
- Resource Exhaustion — Other parts of the application cannot get connections.
- Application Instability — Eventually leads to OutOfMemoryError or connection timeouts.

Best Practice: Always close connection in a finally block or use try-with-resources.

Example: `try (Connection con = DriverManager.getConnection(url, user, pass)) { ... }`

### Q26. What is the difference between `execute()`, `executeQuery()`, and `executeUpdate()`?

#### Answer:

##### `execute(String sql):`

- Executes any SQL statement (DDL, DML, DQL).
- Returns boolean — true for ResultSet, false for update count.

##### `executeQuery(String sql):`

- Executes only SELECT statements.
- Returns a ResultSet object with query results.

##### `executeUpdate(String sql):`

- Executes only DML — INSERT, UPDATE, DELETE.
- Returns int — number of rows affected.

### Q27. What is the difference between Statement and PreparedStatement for repeated execution?

#### Answer:

##### Statement:

- SQL is compiled every time it is executed.
- No pre-compilation — slower for repeated queries.
- Values must be concatenated manually using string operations.

##### PreparedStatement:

- SQL is compiled once and cached (precompiled).
- For repeated execution with different values, it is significantly faster.
- Uses setter methods (`setInt`, `setString`) for clean parameter binding.
- Recommended for loops and batch inserts/updates.

## Q28. How does JDBC handle NoSQL databases?

### Answer:

JDBC is primarily designed for relational (SQL) databases like MySQL and Oracle.

- Most NoSQL databases do not use traditional SQL and have their own Java drivers.

However:

- Some NoSQL databases like MongoDB and Apache Cassandra provide JDBC drivers or connectors through third-party libraries.
- These allow Java applications to communicate using JDBC-like interfaces.

For most NoSQL databases, it is recommended to use their official Java APIs instead of JDBC.